

VISUALIZATION

Visualization is the process of representing data or information in graphical or pictorial form. It helps simplify complex data, making patterns, trends, and insights easier to understand. Using visual elements like charts, graphs, and maps, visualization enhances data analysis, decision-making, and communication in fields such as science, business, and technology.

MATPLOTLIB

Matplotlib is a powerful Python library used for creating static, animated, and interactive visualizations. It provides extensive tools to generate various plots such as line, bar, scatter, and pie charts. Known for its flexibility and customization, Matplotlib helps users produce publication-quality graphs for scientific research and data analysis. A plot of matplotlib contains:

PYLOT

Pyplot is a submodule of Matplotlib that provides simple functions for creating and customizing plots easily. It offers commands for titles, labels, legends, and displaying graphs.

We import pyplot from matplotlib as following:

```
1 import matplotlib.pyplot as plt
```

We also import numpy as a support for the matplotlib :

```
import numpy as np
```

Some of the plots in matplotlib:

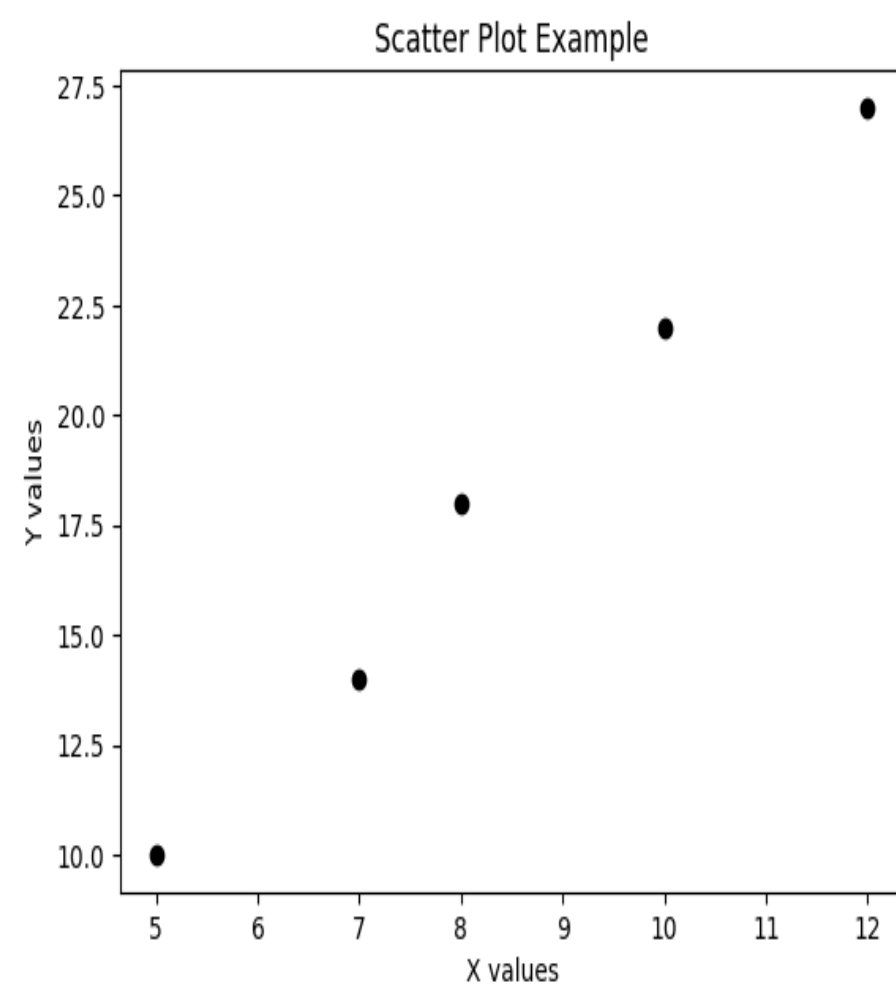
LINE CHART

In Matplotlib, a line chart is created using the `plt.plot()` function. It connects data points with lines to show trends.

Code snippet:

```
[3]: x = [5, 7, 8, 10, 12]
     y = [10, 14, 18, 22, 27]
```

```
plt.scatter(x, y, color='black')
plt.title("Scatter Plot Example")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```



Description:

`np.array()` will generate an array in the specified range.

`plot()` for the plotting the line chart.

`title()` for defining the title, `ylabel()` and `xlabel ()` for labelling y and x axis respectively.

`show()` displays the graph.

BARGRAPH

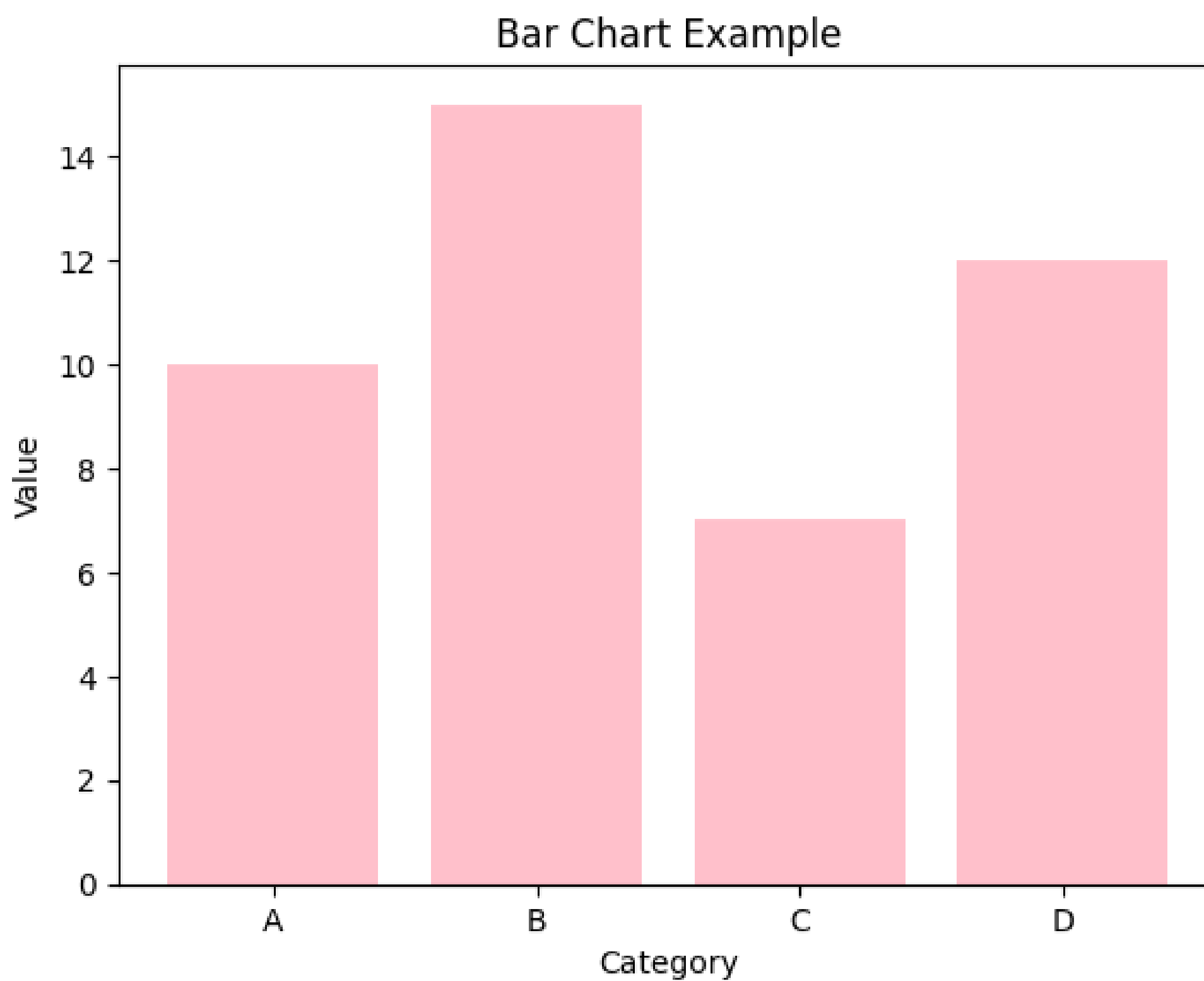
In Matplotlib, a bar graph is created using the `plt.bar()` function. It represents categorical data with rectangular bars showing the value of each category. `bar()` is used for plotting bar graphs, it has parameters like x, y, width, color.

`bar(x, y, color, width)`

Code snippet:

```
[5]: categories = ['A', 'B', 'C', 'D']  
     values = [10, 15, 7, 12]  
  
     plt.bar(categories, values, color='pink')  
     plt.title("Bar Chart Example")  
     plt.xlabel("Category")  
     plt.ylabel("Value")  
     plt.show()
```

Output:



Description:

`random.rand(5)` generates a random array.

Here `bar()` is used along with the parameters, width which denoted the width of the rectangular bars and color for the bars.

HISTOGRAM

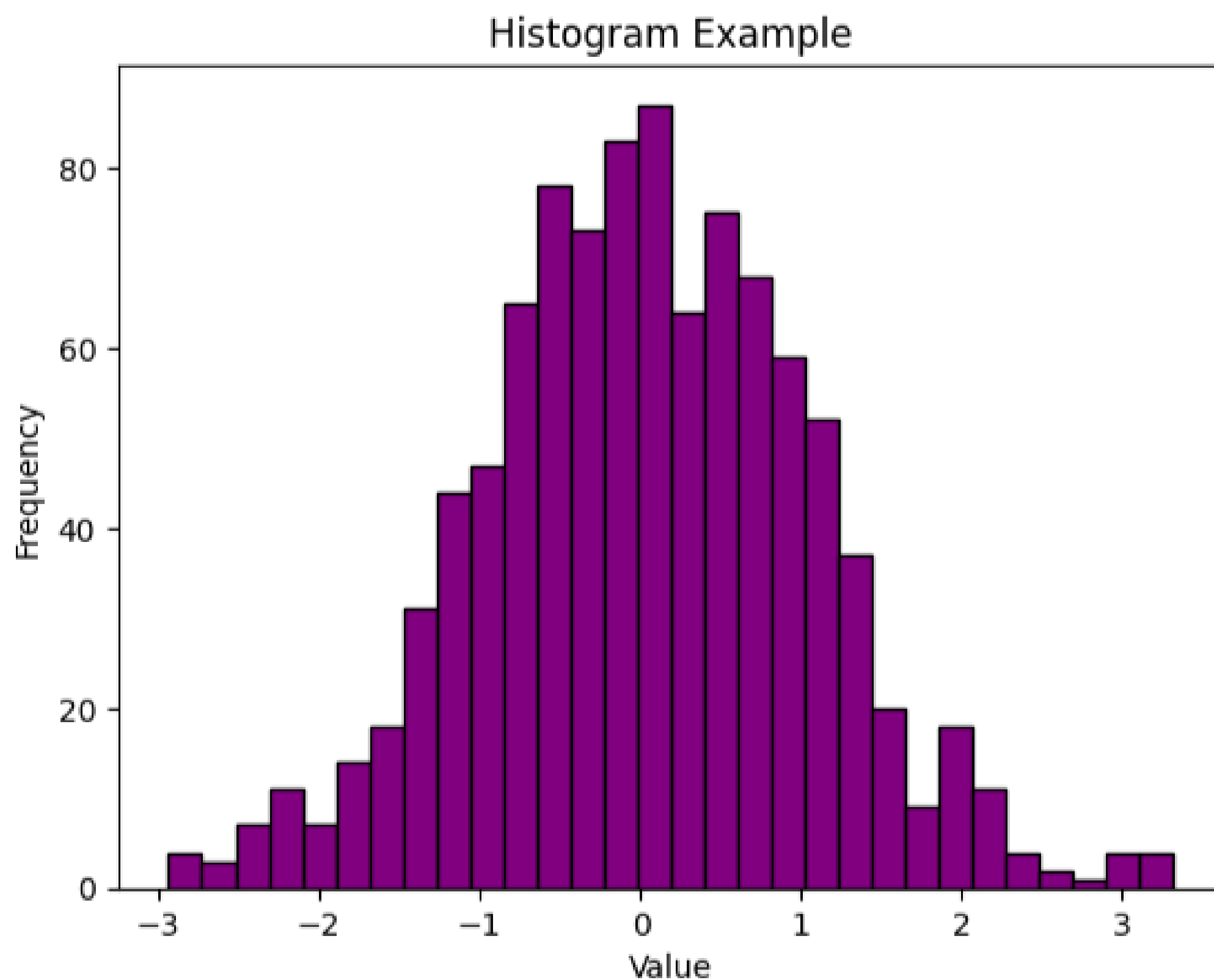
In Matplotlib, a histogram is created using the `plt.hist()` function. It shows the distribution of numerical data by grouping values into bins.

Code snippet:


```
[6]: import numpy as np

data = np.random.randn(1000)
plt.hist(data, bins=30, color='purple', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Output:



Description:

hist() have parameters x which is a random generated array around one hundred with standard deviation 50 of 100 values.

bins indicate the number of sections on x axis.

color indicates the color of rectangular bars.

edge color that divides the rectangular bars.

SCATTER PLOT

In Matplotlib, a scatter plot is created using the plt.scatter() function. It displays the relationship between two numerical variables using dots.

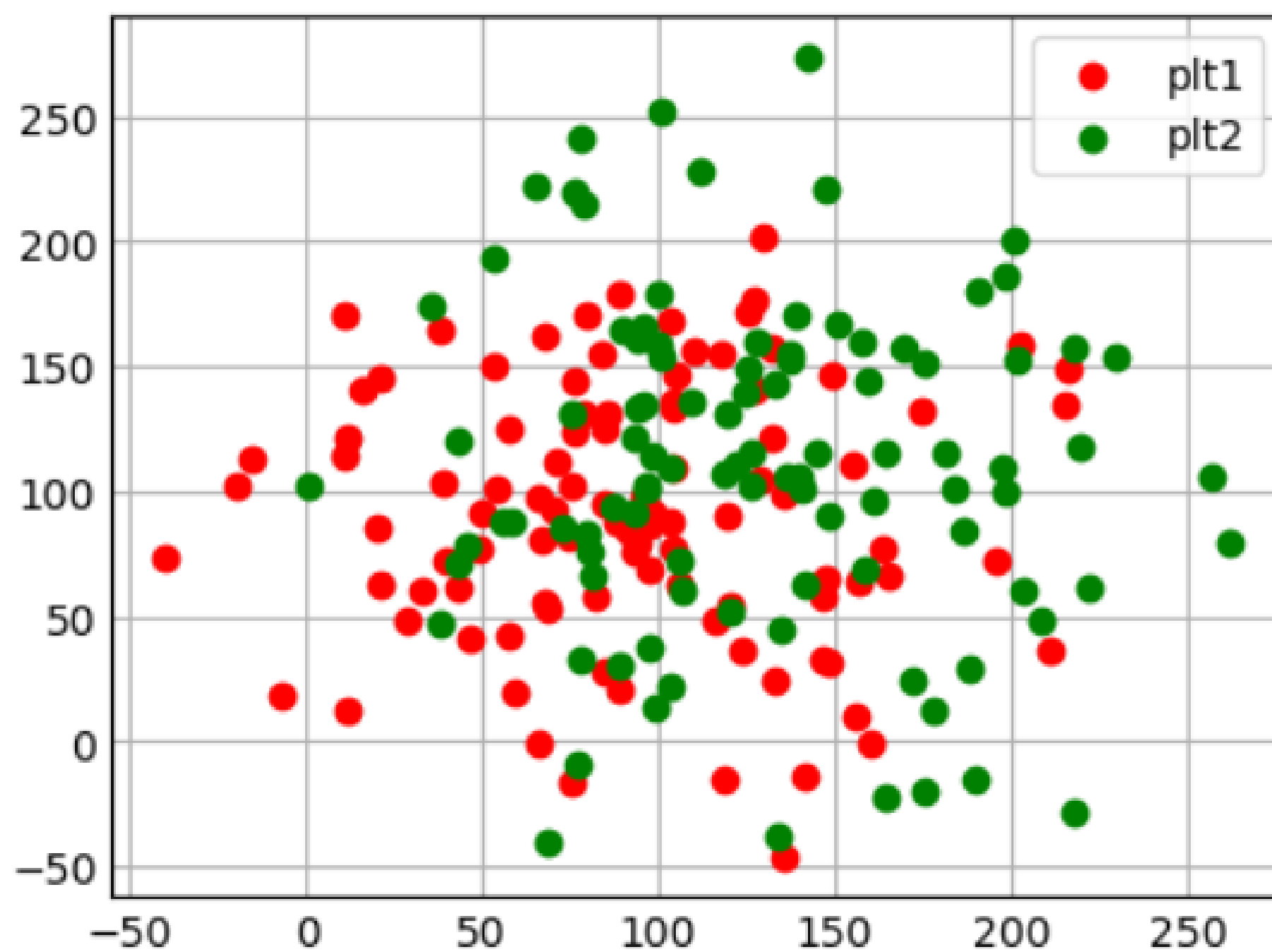
Code snippet:

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x1=np.random.normal(100,50,100)
5 y1=np.random.normal(100,50,100)
6 x2=np.random.normal(120,60,100)
7 y2=np.random.normal(120,60,100)
8 fig = plt.figure(figsize = (4, 3))
9 plt.scatter(x1,y1,c="r")
10 plt.scatter(x2,y2,c="g")
11 plt.legend(["plt1","plt2"])
12
13 plt.show()

```

Output:



Description:

x1,y1 belong to one dataset and x2,y2 belong to another dataset.

Here the scatter() is used for the scatter plot and the parameters x, y and c which represents color.

Legend adds the label which helps to differentiate the plots.

PIECHART

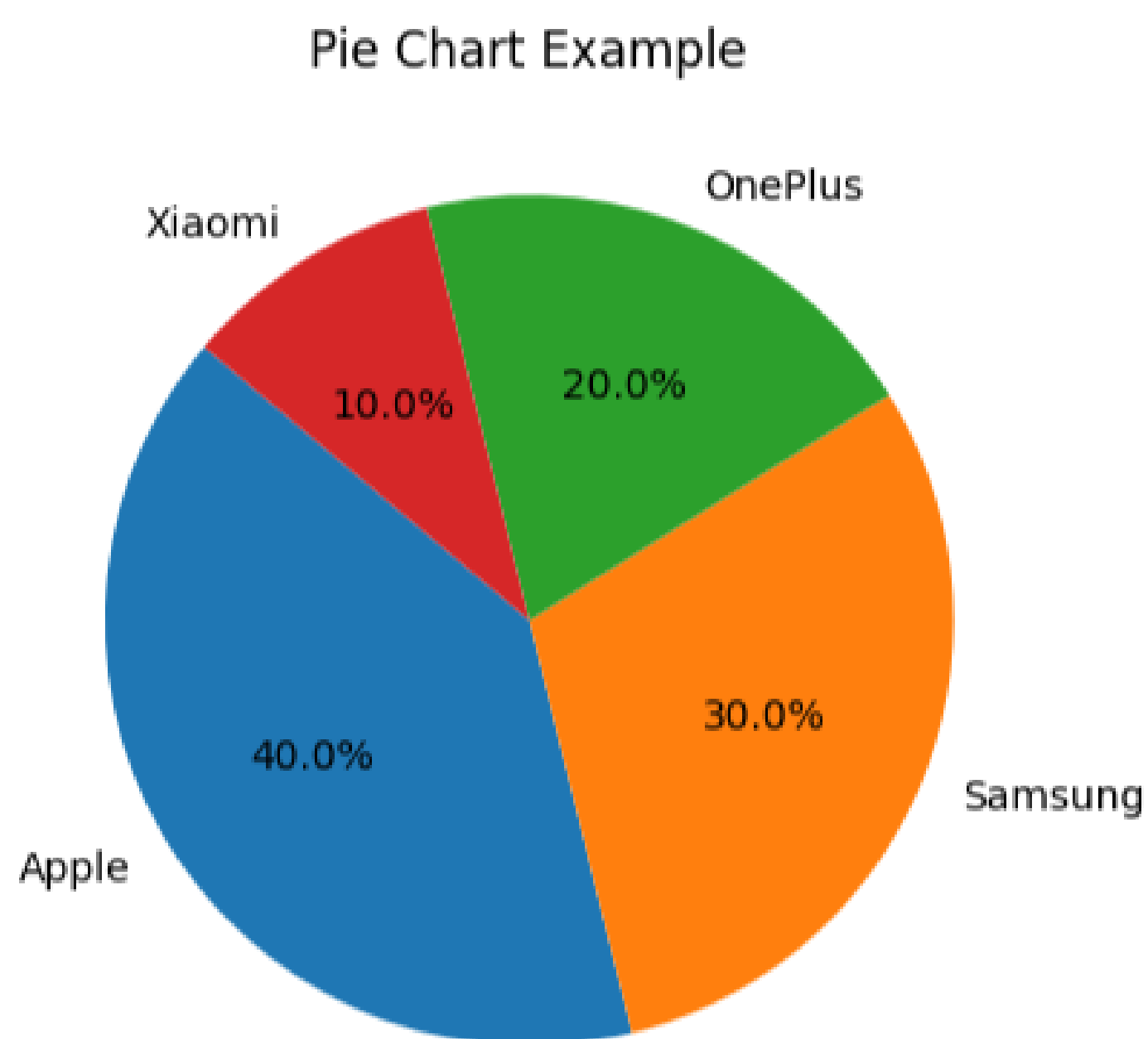
In Matplotlib, a pie chart is created using the plt.pie() function. It shows how different parts make up a whole, represented as slices of a circle.

Code snippet:

```
[7]: labels = ['Apple', 'Samsung', 'OnePlus', 'Xiaomi']
     sizes = [40, 30, 20, 10]

     plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
     plt.title("Pie Chart Example")
     plt.show()
```

Output:



Description:

Here in the code, `size` is the x variable and labels are defined using `labels` keyword.

`wedgeprops` attribute is used to define the `linewidth` and the `color` to separate the elements in the pie chart.

`color` array contains the colors to each element.

SEABORN

Seaborn is a high-level Python visualization library built on top of Matplotlib. It simplifies the creation of attractive, informative, and statistical graphics. With beautiful default themes and easy integration with Pandas, Seaborn supports advanced plots like heatmaps, pair plots, and violin plots, making data analysis and pattern visualization effortless.

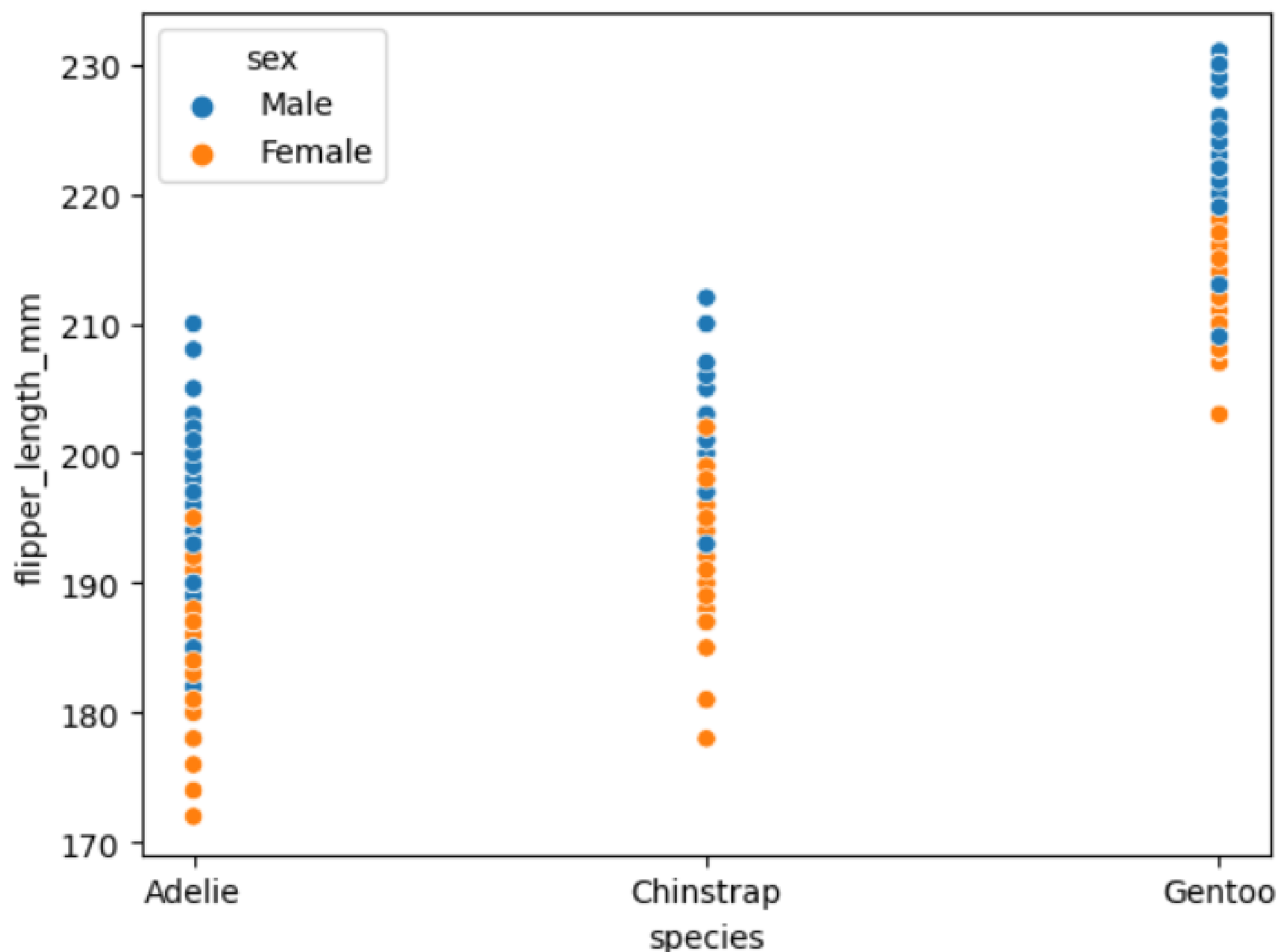
Here are some sample codes for some of the graphs.

Scatter plot:

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 df = sns.load_dataset("penguins")
```

```
1 sns.scatterplot(y='flipper_length_mm', x='species', hue='sex', data=df)
2 plt.show()
```

Output:



Description:

Here the different colors on sex is due to the parameter hue. Since hue is on “ sex” column there is a difference for male and female.

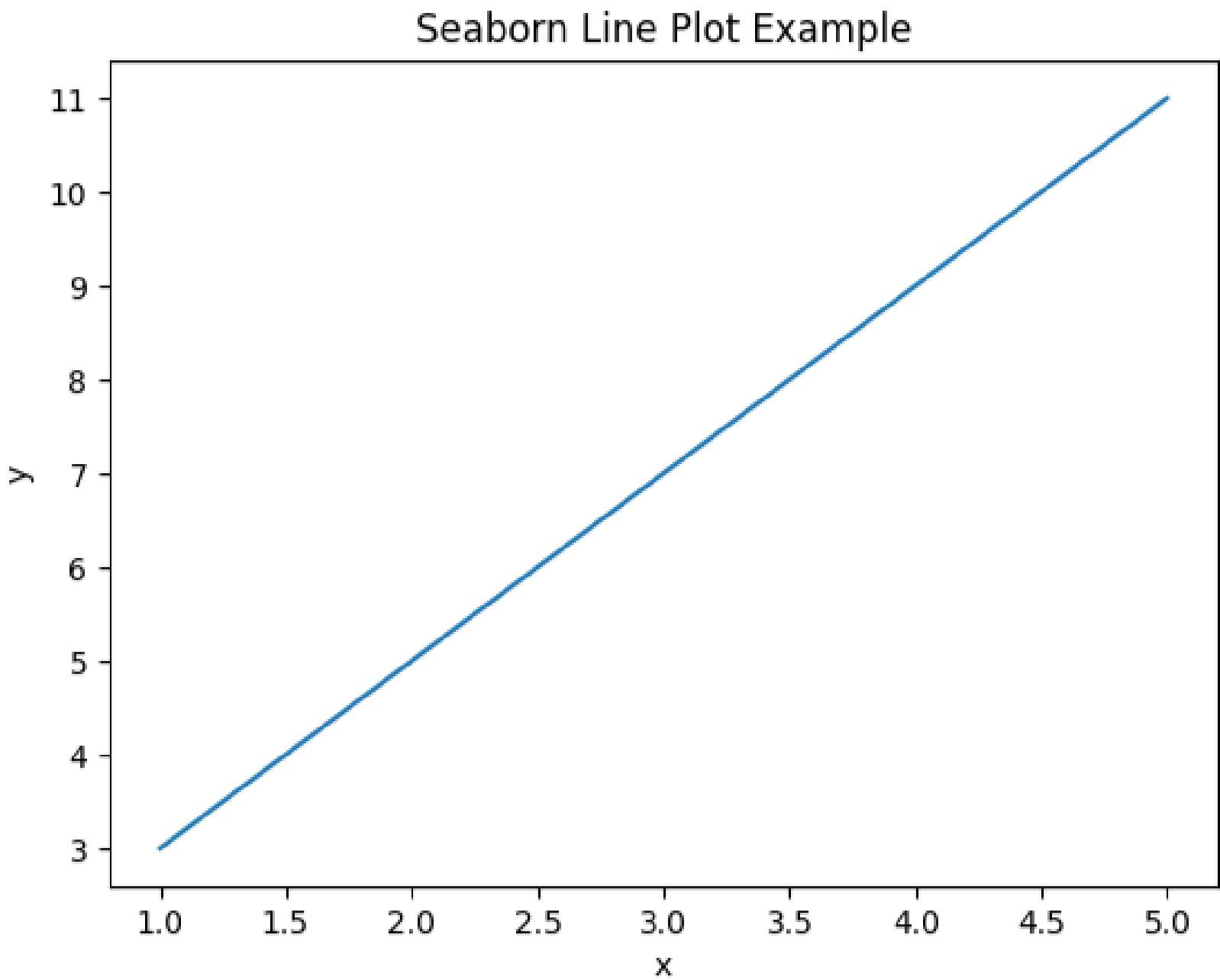
LINE PLOT

```
[8]: import seaborn as sns
import pandas as pd

data = pd.DataFrame({
    'x': [1, 2, 3, 4, 5],
    'y': [3, 5, 7, 9, 11]
})

sns.lineplot(x='x', y='y', data=data)
plt.title("Seaborn Line Plot Example")
plt.show()
```

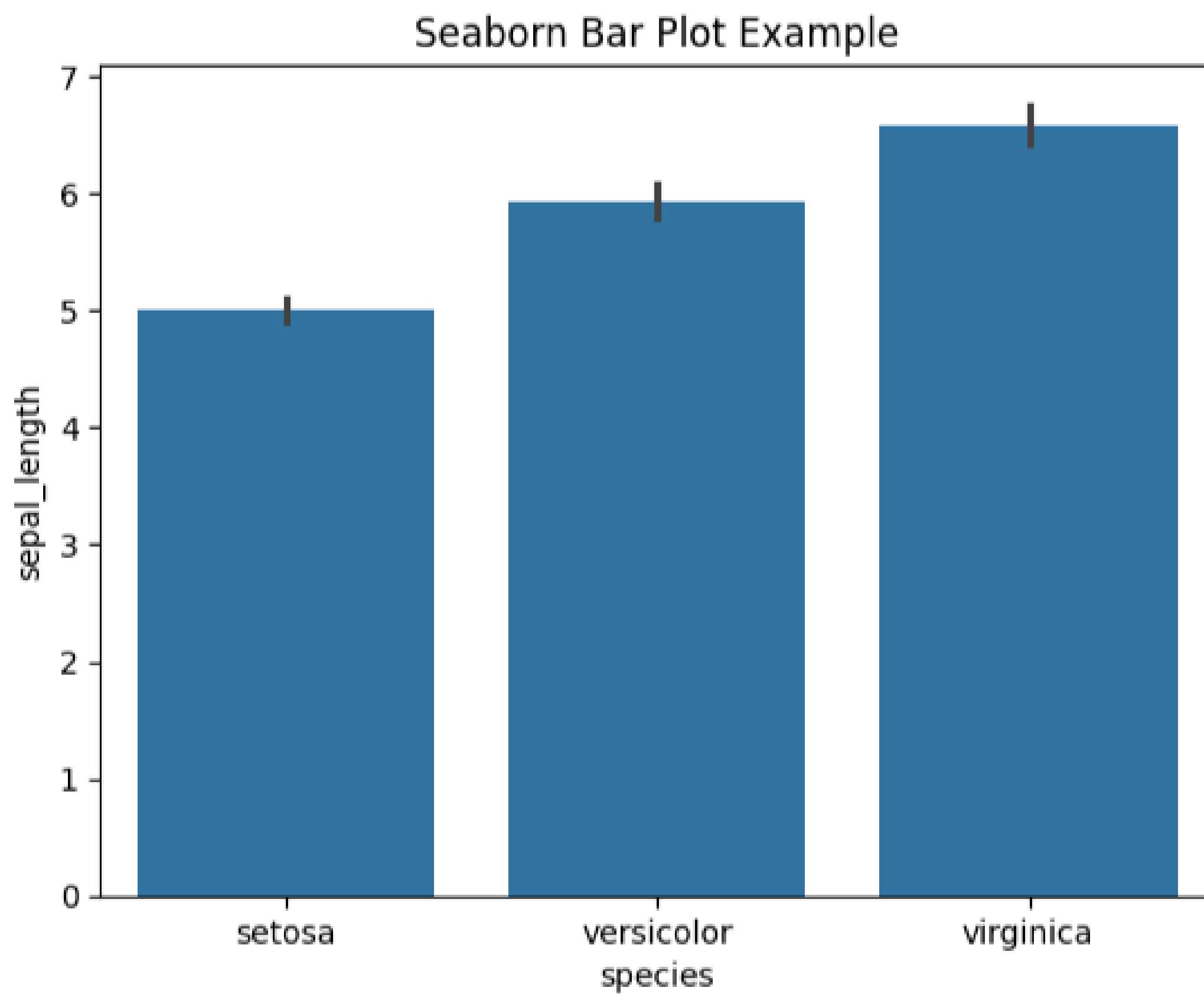
Output:



BAR PLOT

```
[10]: sns.barplot(x="species", y="sepal_length", data=iris)
plt.title("Seaborn Bar Plot Example")
plt.show()
```

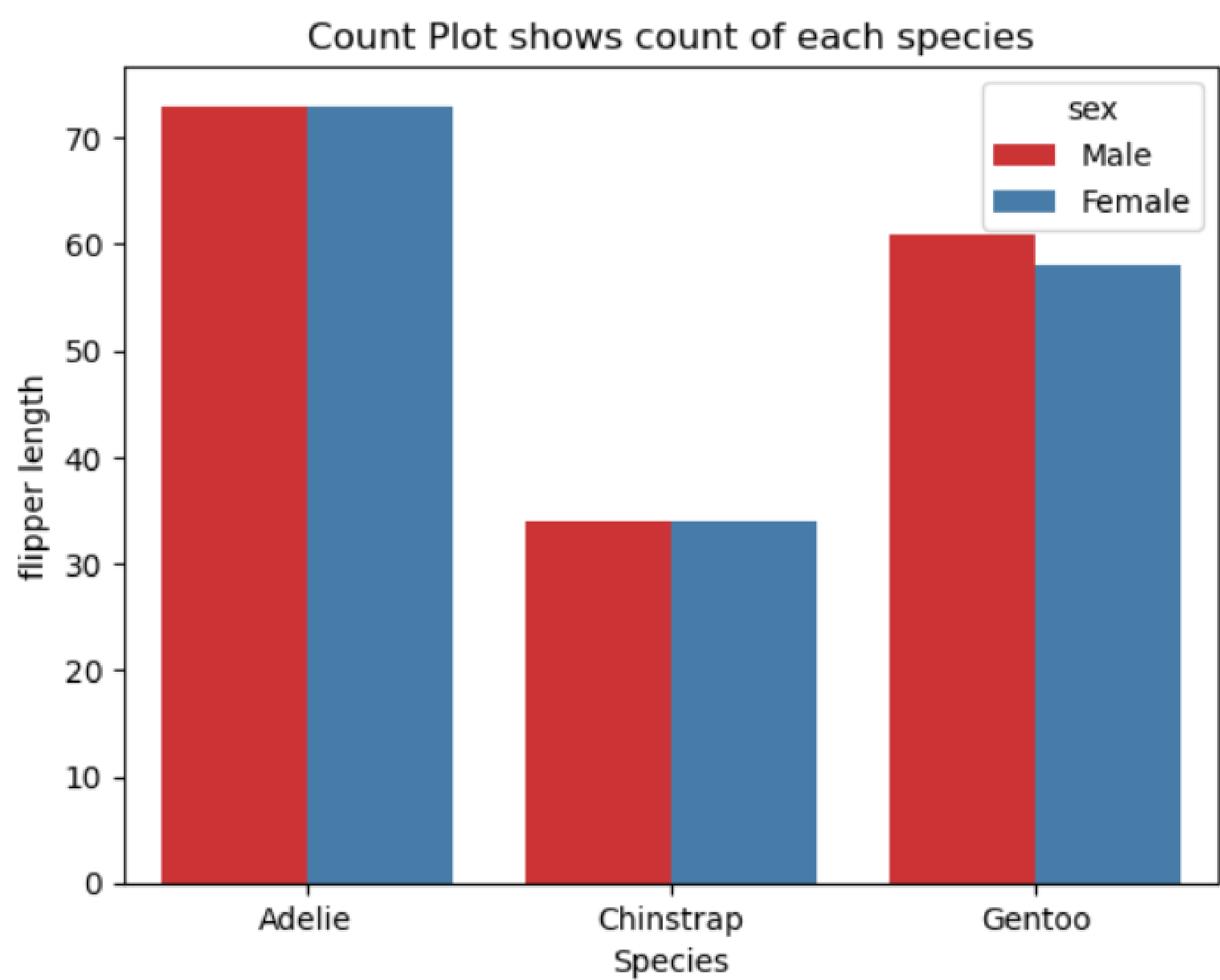

Output:



COUNT PLOT

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import seaborn as sns
4
5 sns.countplot(x='species',hue='species', data=df, palette="Set1")
6 plt.title('Count Plot shows count of each species')
7 plt.ylabel('flipper length')
8 plt.xlabel('Species')
9 plt.show()
```

Output:



Description:

The count plots give the count of the data passed.

Here x is the species, hue is sex and palette as Set1.

Palette sets the color and it is Set1 is the default palette in seaborn.

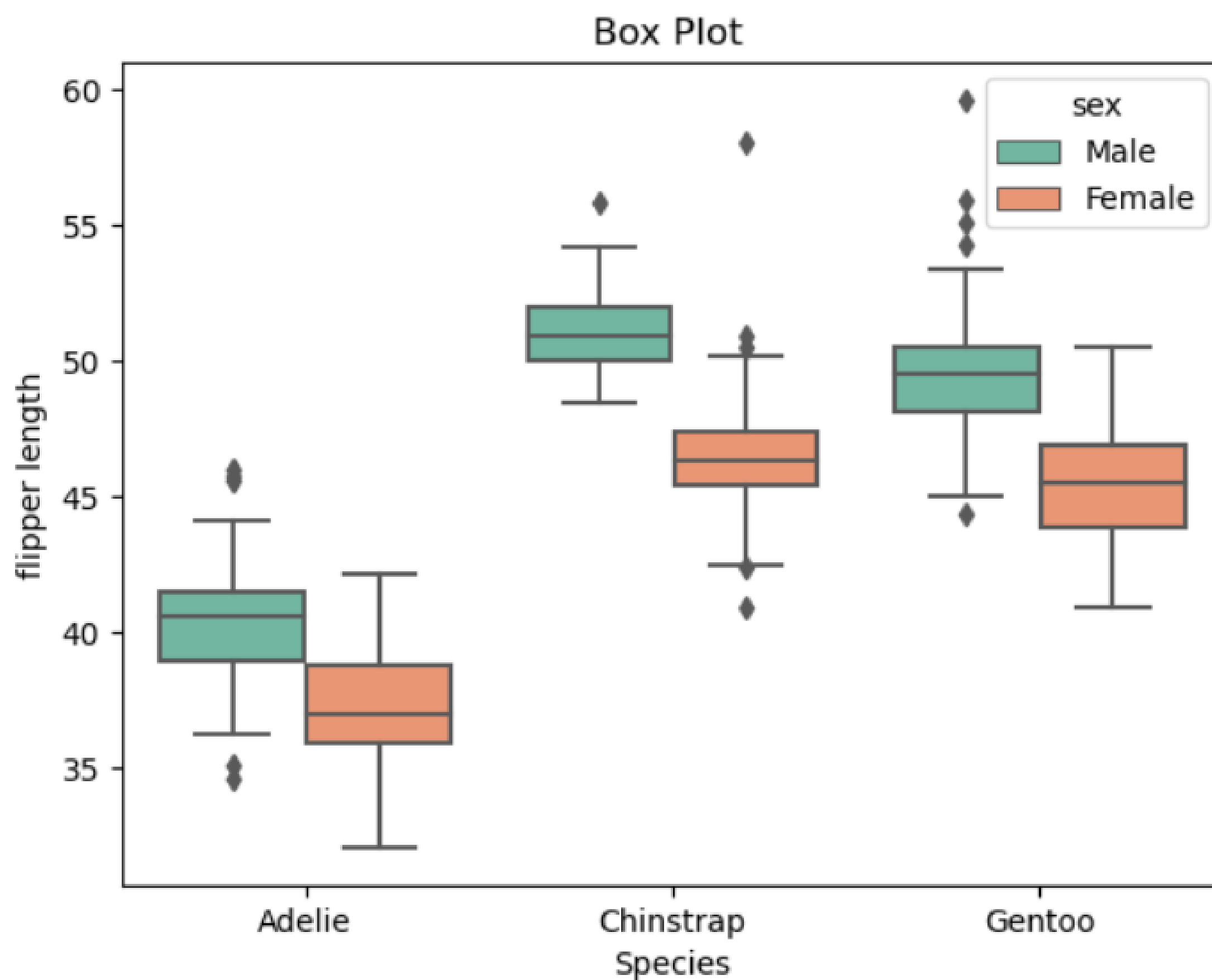
BOX PLOT

Box plot shows the quartiles where the data lies in interquartile range will be inside the box. The points which are away from the whiskers are outliers.

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import seaborn as sns
4
5 sns.boxplot(x='species',y="bill_length_mm",hue='sex', data=df, palette="Set2")
6 plt.title('Box Plot ')
7 plt.ylabel('flipper length')
8 plt.xlabel('Species')
9 plt.show()
```

Output:



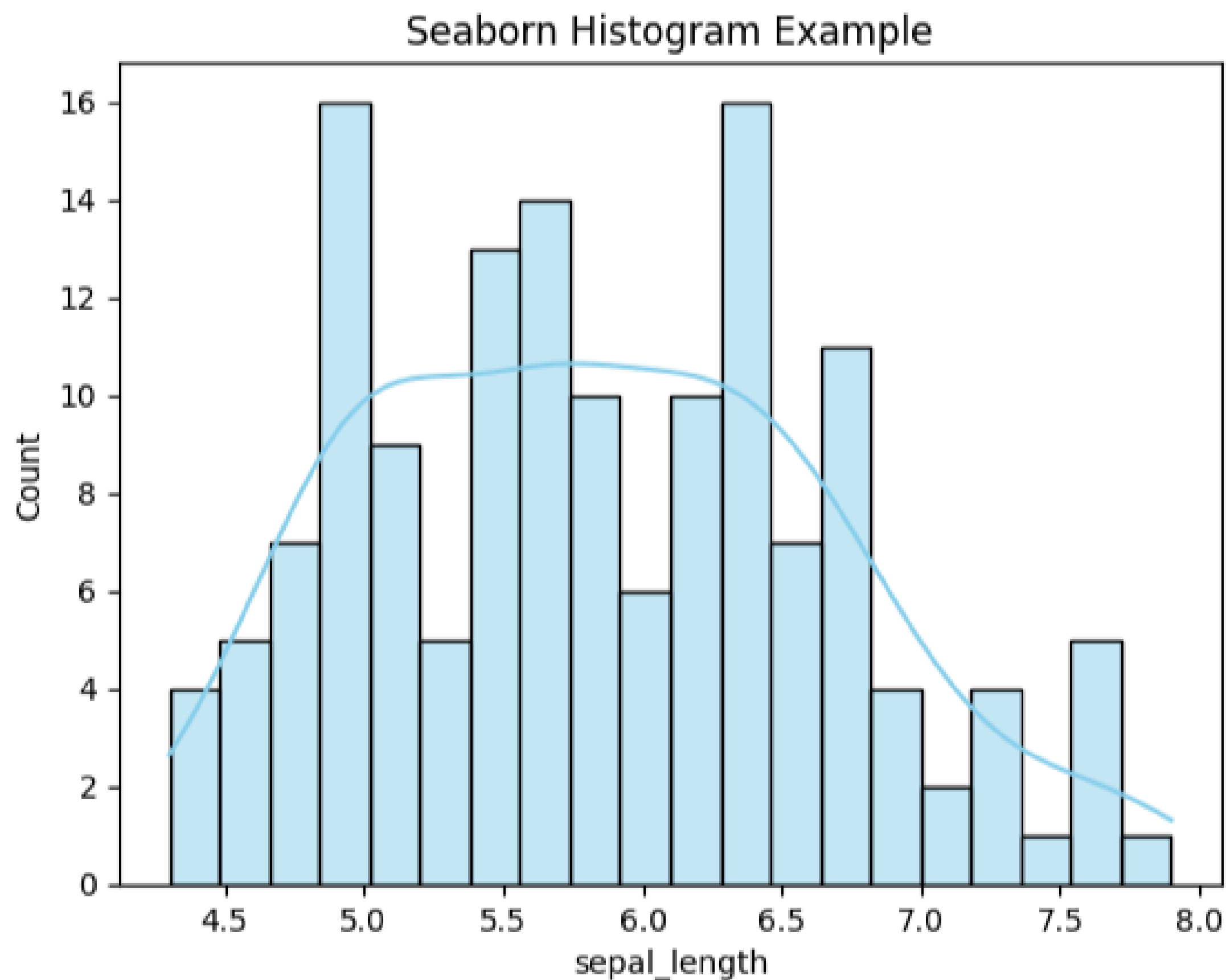
Description:

The parameter x should be numeric as the plot shows the difference between categories.

HISTO PLOT

```
[11]: sns.histplot(iris['sepal_length'], bins=20, kde=True, color='skyblue')
plt.title("Seaborn Histogram Example")
plt.show()
```

Output:



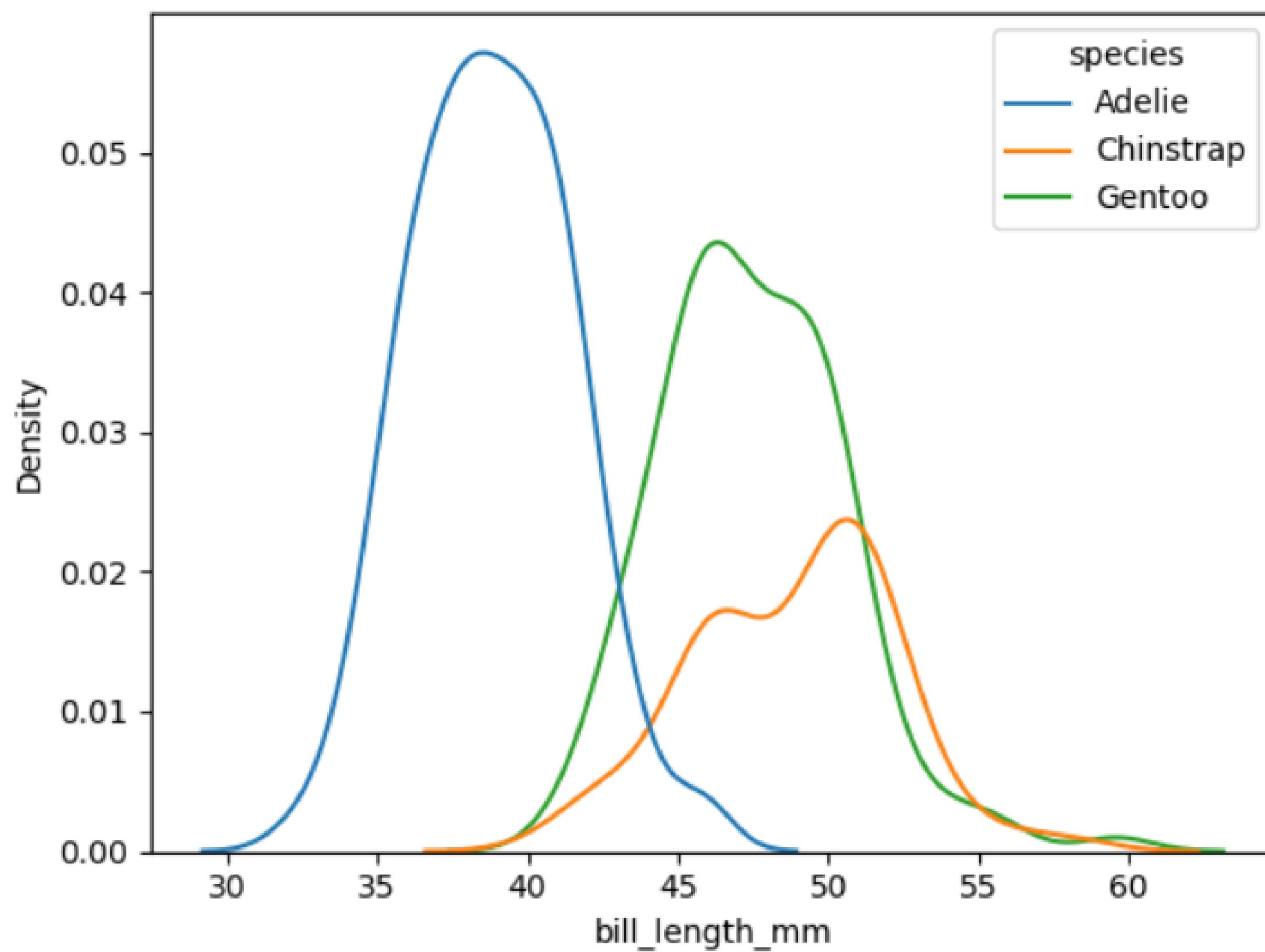
KDE PLOT

KDE stands for kernel density estimate. It creates a curve as based on probability. It creates single graph for multiple data samples.

Code snippet:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import seaborn as sns
4
5 sns.kdeplot(x='bill_length_mm',hue='species',data=df )
6 plt.show()
```

Output:

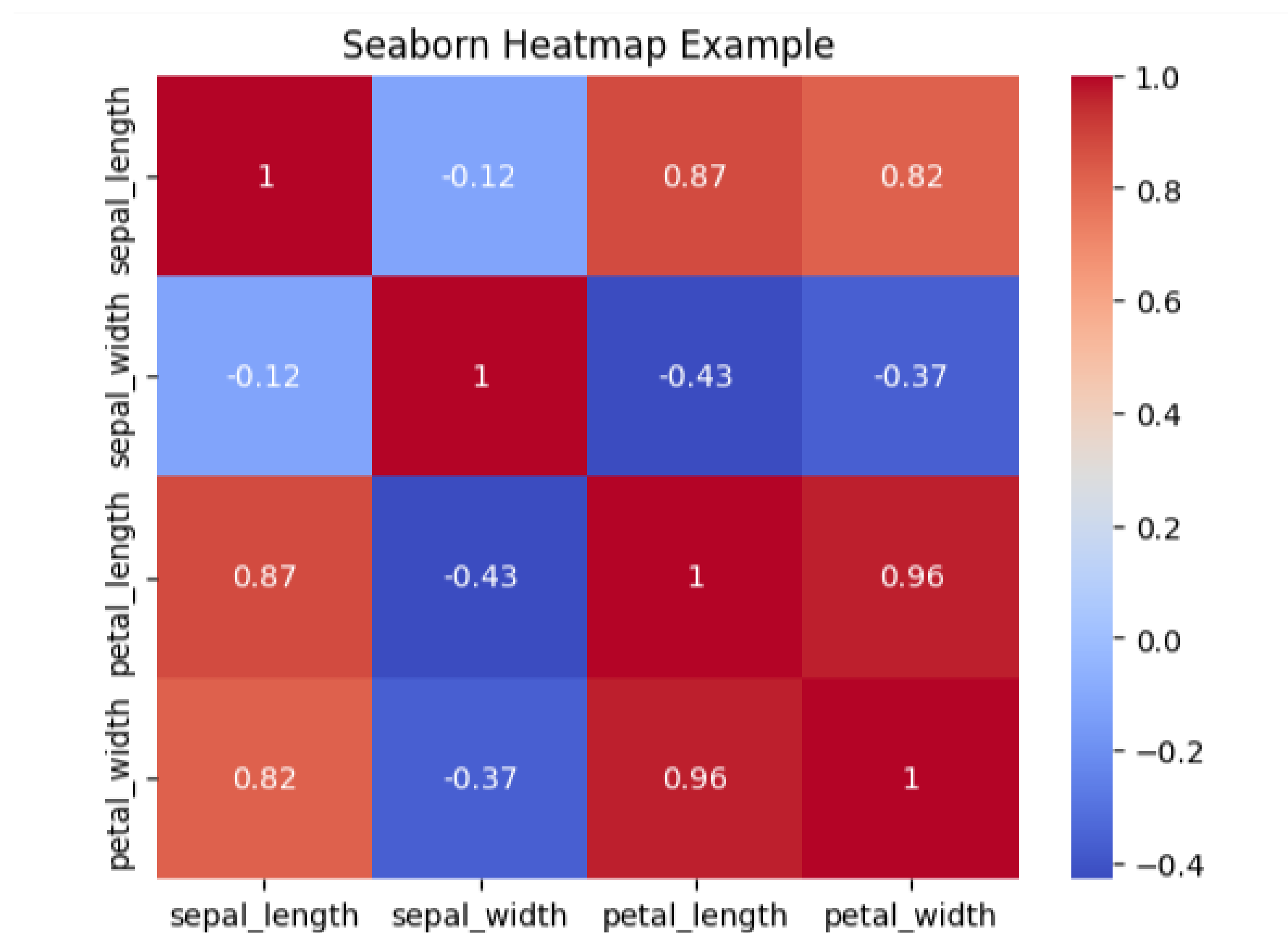


HEATMAP

Heatmaps uses correlation matrix and visualizes data. The datapoints where the higher values get brighter colors and lower values get darker colors.

Code snippet:

```
[12]: corr = iris.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Seaborn Heatmap Example")
plt.show()
```



Output

REGPLOT

Regression plots shows the relationship between variables along with the regression line.

Code snippet:

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.regplot(x="bill_length_mm", y="flipper_length_mm", data=penguins)
5 plt.title("Regression Plot of Bill Length vs. Flipper Length")
6 plt.xlabel("Bill Length (mm)")
7 plt.ylabel("Bill Depth (mm)")
8 plt.show()
9
```

Output:

Regression Plot of Bill Length vs. Flipper Length

